

Week04 实验报告

mrw1114

September, 14th

1 个人信息

- 姓名：王俊杰
- 学号：25020007122
- Github 仓库链接：<https://github.com/mrw1114/fundamentals-of-system-dev-tools>

2 课上实验

2.1 第 13 题：建立可执行的本地质量门禁

2.1.1 复制项目并修改配置文件

在 q10 目录的上级执行 `cp -r ./q10 q13` 复制项目。随后进入 q13 目录，修改 `pyproject.toml` 文件，加入 `ruff` 和 `pytest` 的最小配置，配置了 `line-length`（单行最大字符数，默认 88）、`target-version`（项目版本）以及 `testpaths`（`pytest` 寻找测试文件的默认路径）。

```

mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test$ cp -r ./q10 q13
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test$ ls q13
ai_log.md  dist  pyproject.toml  src  test  venv
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test$ cd q13
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q13$ vim pyproject.toml
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q13$ cat pyproject.toml
# pyproject.toml
[build-system]
requires = ["setuptools>=68"]
build-backend = "setuptools.build_meta"
[project]
name = "greetlab-25020007122"
version = "0.1.1"
[project.scripts]
sdt-greet = "greetlab.cli:main"
[tool.ruff]
line-length = 88 # 指定单行最多的字符数，约定俗成为88个字符。
target-version = "py312" # 指定项目使用的 python 版本
[tool.pytest.ini_options]
testpaths = ["test"] # 指定 pytest 的默认路径
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q13$

```

图 1: 在 pyproject.toml 中添加 ruff 和 pytest 的最小配置

2.1.2 补充测试用例

新建 test/test_cli.py 文件，补充正常姓名和空白姓名测试，正常姓名测试中 stdout 应有”Hello, 王俊杰”，空白姓名测试返回码应为 2。

```

mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q13$ vim test/test_cli.py
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q13$ cat test/test_cli.py
import subprocess
import sys
import pytest

def test_whitespace_name_exits_with_2():
    cmd = ["sdt-greet", "--name", " "]
    result = subprocess.run(cmd, capture_output=True, text=True)
    # 预期退出码为 2
    assert result.returncode == 2

def test_normal_name():
    cmd = ["sdt-greet", "--name", "王俊杰"]
    result = subprocess.run(cmd, capture_output=True, text=True)
    assert result.returncode == 0
    assert "Hello, 王俊杰" in result.stdout
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q13$

```

图 2: test_cli.py 中的测试内容

2.1.3 运行质量检查并修复问题

执行 `ruff format` 和 `ruff check`, 发现 EXE002 (缺少 shebang)、I001 (导入未排序)、F401 (未使用导入)、PLW1510 (`subprocess.run` 缺少 `check` 参数) 等错误。运行 `pytest` 时, `test_normal_name` 测试失败。随后执行 `ruff check -fix` 自动修复部分问题, 但 EXE002 和 PLW1510 无法自动修复: EXE002 需要手动移除文件可执行权限或添加 shebang; PLW1510 需要人工判断 `subprocess.run` 的 `check` 参数应设为 `True` 还是 `False` (设置为 `True` 时, `subprocess.run` 若有非 0 返回码则抛出异常)。手动修改 `test/test_cli.py`, 添加 `check=False` 参数; 随后鉴于包文件和测试文件无需直接执行, 使用 `chmod -x` 移除源文件和测试文件的可执行权限。再次运行 `ruff check`, 显示全部通过。最后编写 `check.sh` 脚本, 依次执行 `ruff format --check`、`ruff check` 和 `pytest`, 使用 `set -e` 确保出错时立即退出, 并赋予执行权限。

```

(venv) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q13$ ruff format
2 files reformatted, 2 files left unchanged
(venv) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q13$ ruff check
EXE002 The file is executable but no shebang is present
--> src/greetlab/__init__.py:1:1

EXE002 The file is executable but no shebang is present
--> src/greetlab/cli.py:1:1

EXE002 The file is executable but no shebang is present
--> test/test_cli.py:1:1

I001 [*] Import block is un-sorted or un-formatted
--> test/test_cli.py:1:1
|
1 | / import subprocess
2 | | import sys
3 | | import pytest
  | | _____^
help: Organize imports
|
2 | import sys
3 +
4 | import pytest
|

F401 [*] 'sys' imported but unused
--> test/test_cli.py:2:8
|
1 | import subprocess
2 | import sys
  |         ^^^
3 | import pytest
|
help: Remove unused import: 'sys'
|
1 | import subprocess
  - import sys
2 | import pytest
|

F401 [*] 'pytest' imported but unused
--> test/test_cli.py:3:8
|
1 | import subprocess
2 | import sys
3 | import pytest
  |         ^^^^^^
help: Remove unused import: 'pytest'

```

图 3: 运行 ruff format 和 ruff check 的前半段输出

```

2 | import sys
  | - import pytest
3 |
  |
PLW1510 `subprocess.run` without explicit `check` argument
--> test/test_cli.py:8:14
  |
6 | def test_whitespace_name_exits_with_2():
7 |     cmd = ["sdt-greet", "--name", " "]
8 |     result = subprocess.run(cmd, capture_output=True, text=True)
  |             ^^^^^^^^^^^^^^^^^
9 |     # 预期退出码为 2
10 |    assert result.returncode == 2
  |
help: Add explicit `check=False`

PLW1510 `subprocess.run` without explicit `check` argument
--> test/test_cli.py:15:14
13 | def test_normal_name():
14 |     cmd = ["sdt-greet", "--name", "王俊杰"]
15 |     result = subprocess.run(cmd, capture_output=True, text=True)
  |             ^^^^^^^^^^^^^^^^^
16 |     assert result.returncode == 0
17 |     assert "Hello, 王俊杰" in result.stdout
  |
help: Add explicit `check=False`

Found 8 errors.
[*] 3 fixable with the `--fix` option.
(mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q13$ pytest
===== test session starts =====
platform linux -- Python 3.12.3, pytest-9.1.1, pluggy-1.6.0
rootdir: /home/mrw/sys-dev-test/q13
configfile: pyproject.toml
testpaths: test
collected 2 items

test/test_cli.py .. [100%]

```

图 4: ruff check 的后半段输出与 pytest 失败结果

```
(venv) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q13$ ruff check --fix
EXE002 The file is executable but no shebang is present
--> src/greetlab/__init__.py:1:1

EXE002 The file is executable but no shebang is present
--> src/greetlab/cli.py:1:1

EXE002 The file is executable but no shebang is present
--> test/test_cli.py:1:1

PLW1510 `subprocess.run` without explicit `check` argument
--> test/test_cli.py:6:14
4 | def test_whitespace_name_exits_with_2():
5 |     cmd = ["sdt-greet", "--name", " "]
6 |     result = subprocess.run(cmd, capture_output=True, text=True)
  |               ^^^^^^^^^^^^^^^^^
7 |     # 预期退出码为 2
8 |     assert result.returncode == 2
  |
help: Add explicit `check=False`

PLW1510 `subprocess.run` without explicit `check` argument
--> test/test_cli.py:13:14
11 | def test_normal_name():
12 |     cmd = ["sdt-greet", "--name", "王俊杰"]
13 |     result = subprocess.run(cmd, capture_output=True, text=True)
  |               ^^^^^^^^^^^^^^^^^
14 |     assert result.returncode == 0
15 |     assert "Hello, 王俊杰" in result.stdout
  |
help: Add explicit `check=False`

Found 9 errors (4 fixed, 5 remaining).
```

图 5: 使用 `ruff check --fix` 自动修复后剩余的无法自动修复的问题

```
(venv) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q13$ vim test/test_cli.py
(venv) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q13$ chmod -x src/greetlab/__init__.py src/greetlab/cli.py test/test_cli.py
(venv) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q13$ ruff check
All checks passed!
(venv) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q13$
```

图 6: 修复剩余问题后 `ruff check` 通过

2.1.4 运行 `check.sh` 验证

编写并执行 `./check.sh`，所有检查通过并成功返回 0。

```
(venv) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q13$ vim check.sh
(venv) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q13$ cat check.sh
#!/bin/bash
set -e
ruff format --check
ruff check
pytest

echo "所有检查通过"
(venv) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q13$ chmod +x check.sh
(venv) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q13$ ./check.sh
4 files already formatted
All checks passed!
===== test session starts =====
platform linux -- Python 3.12.3, pytest-9.1.1, pluggy-1.6.0
rootdir: /home/mrw/sys-dev-test/q13
configfile: pyproject.toml
testpaths: test
collected 2 items

test/test_cli.py .. [100%]

===== 2 passed in 0.05s =====
所有检查通过
(venv) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q13$
```

图 7: check.sh 的编写与执行成功

2.2 第 14 题：让 Make 只重建真正受影响的产物

2.2.1 创建源文件

在 q14 目录下创建给定的 data.csv、stats.py、build_report.py 和 report.md 文件。

```

mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q14$ vim data.csv
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q14$ vim stats.py
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q14$ vim build_report.py
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q14$ vim report.md
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q14$ cat data.csv
name,value
alpha,2
beta,3
gamma,5
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q14$ cat stats.py build_report.py
# stats.py
import csv
with open("data.csv") as f:
    total = sum(int(r["value"]) for r in csv.DictReader(f))
open("stats.txt", "w").write(str(total))
# build_report.py
text = open("report.md").read()
total = open("stats.txt").read()
open("report.txt", "w").write(f"{text}\nTotal: {total}\n")
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q14$ cat report.md
# Data Report
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q14$

```

图 8: 创建给定的源文件

2.2.2 编写 Makefile

编写 Makefile，包含 all、stats.txt、report.txt 和 clean 目标，列出数据文件、脚本和中间产物的依赖关系。其中.PHONY 声明表示将对应目标设置为伪目标，因为 make 优先将目标视为对应的文件，而 all 表示构建系统默认的目标，其依赖表示构建系统默认构建的产物；clean 无依赖也不生成产物，其动作用于清除构建过程中的中间产物。如果不进行.PHONY 声明，则当待构建的目录中存在名称为 all 或 clean 的文件时，可能无法正确执行 make 和 make clean 等命令。

```

mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q14$ vim Makefile
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q14$ cat Makefile
.PHONY: all clean

all: report.txt

stats.txt: data.csv stats.py
    python3 stats.py

report.txt: stats.txt build_report.py report.md
    python3 build_report.py

clean:
    rm -f stats.txt report.txt
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q14$

```

图 9: Makefile 内容

2.2.3 验证构建逻辑

首次执行 make，成功生成 stats.txt 和 report.txt。第二次执行 make，未执行配方更新产物。执行 touch data.csv 后再次 make，发现 stats.txt 和 report.txt 重新生成。执行 make clean，仅删除生成物而源文件不受影响。

```
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q14$ make
python3 stats.py
python3 build_report.py
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q14$ ls -l
总计 28
-rw-rw-r-- 1 mrw mrw 142 Sep 14 03:54 build_report.py
-rw-rw-r-- 1 mrw mrw 34 Sep 14 03:53 data.csv
-rw-rw-r-- 1 mrw mrw 193 Sep 14 03:47 Makefile
-rw-rw-r-- 1 mrw mrw 14 Sep 14 03:40 report.md
-rw-rw-r-- 1 mrw mrw 25 Sep 14 03:56 report.txt
-rw-rw-r-- 1 mrw mrw 151 Sep 14 03:54 stats.py
-rw-rw-r-- 1 mrw mrw 2 Sep 14 03:56 stats.txt
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q14$ cat stats.txt
10mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q14$ cat report.txt
# Data Report

Total: 10
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q14$ make
make: 对“all”无需做任何事。
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q14$ touch data.csv
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q14$ make
python3 stats.py
python3 build_report.py
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q14$ cat report.txt
# Data Report

Total: 10
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q14$ cat stats.txt
10mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q14$ make clean
rm -f stats.txt report.txt
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q14$ ls -l
总计 20
-rw-rw-r-- 1 mrw mrw 142 Sep 14 03:54 build_report.py
-rw-rw-r-- 1 mrw mrw 34 Sep 14 03:58 data.csv
-rw-rw-r-- 1 mrw mrw 193 Sep 14 03:47 Makefile
-rw-rw-r-- 1 mrw mrw 14 Sep 14 03:40 report.md
-rw-rw-r-- 1 mrw mrw 151 Sep 14 03:54 stats.py
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q14$
```

图 10: make 产物验证与更新后的 make clean

2.3 第 15 题：把本地 API 数据转换为可读报告

2.3.1 创建 JSON 数据并启动本地服务

创建 packages.json 文件并填入给定的 JSON 数据。在 q15 目录下运行 python3 -m http.server 8000 启动本地 HTTP 服务。

```
nrw@nrw-VMware-Virtual-Platform:~/sys-dev-test/q14$ cd ../q15
nrw@nrw-VMware-Virtual-Platform:~/sys-dev-test/q15$ vim packages.json
nrw@nrw-VMware-Virtual-Platform:~/sys-dev-test/q15$ cat packages.json
[
  {"name": "alpha", "status": "active", "downloads": 120, "version": "1.2.0"},
  {"name": "beta", "status": "inactive", "downloads": 900, "version": "2.0.0"},
  {"name": "gamma", "status": "active", "downloads": 450, "version": "1.5.1"},
  {"name": "delta", "status": "active", "downloads": 450, "version": "0.9.0"},
  {"name": "epsilon", "status": "active", "downloads": 80, "version": "3.1.0"}
]
```

图 11: 创建 packages.json 文件

2.3.2 编写 api_report.sh 生成报告

编写 api_report.sh, 使用 curl -fsS 获取 JSON 数据, 通过 jq 筛选 status 为 active 且 downloads 不少于 100 的记录, 并使用 sort 按 downloads 降序、name 升序排列。最后通过 awk 格式化为 Markdown 表格, 写入 summary.md。执行脚本并查看生成的报告。

```

mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q15$ python3 -m http.server 8000 &
[1] 6827
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q15$ Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) .
..

mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q15$ vim api_report.sh
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q15$ cat api_report.sh
#!/usr/bin/env bash

data=$(curl -fsS http://127.0.0.1:8000/packages.json)

{
  echo "# Summary for Active Packages"
  echo ""
  echo "| name | version | downloads |"
  echo "|-----|-----|-----|"
  echo "$data" | jq -r '.[] | select(.status == "active" and .downloads >= 100) | [.name, .version, .downloads
] | @tsv' \
    | sort -k3,3nr -k1,1 \
    | awk -F'\t' '{printf "| %s | %s | %s |\n", $1, $2, $3}'
} > summary.md

echo "Report written to summary.md"
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q15$ chmod +x api_report.sh
找不到命令 "chmod"，您的意思是：
  "chmod" 命令来自 Debian 软件包 coreutils (9.4-3ubuntu6.2)
尝试 sudo apt install <deb name>
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q15$ chmod +x api_report.sh
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q15$ ./api_report.sh
127.0.0.1 - - [14/Sep/2026 04:26:39] "GET /packages.json HTTP/1.1" 200 -
Report written to summary.md
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q15$ cat summary.md
# Summary for Active Packages

| name | version | downloads |
|-----|-----|-----|
| delta | 0.9.0 | 450 |
| gamma | 1.5.1 | 450 |
| alpha | 1.2.0 | 120 |
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q15$

```

图 12: 创建服务器并通过 API 获取信息生成 summary.md

2.4 第 16 题：修复并交付一个陌生的小型工具仓库

由于在之前第 10 题，第 13 题中没有为 cli.py 添加保护块，导致 pytest 测试只能对 build 并安装的 whl 包进行测试，所以此处 make check 中包含了构建产物，强制重新安装产物，执行 check.sh，卸载包并删除 build 产物的流程。

2.4.1 复制项目并初始化 Git

执行 `cp -r ./q13 q16` 复制项目，进入 `q16` 目录后执行 `git init` 初始化仓库，并完成初始提交。

```
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test$ cp -r ./q13 q16
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test$ ls -l q16
总计 28
-rwxrwxr-x 1 mrw mrw 366 Sep 14 04:32 ai_log.md
-rwxrwxr-x 1 mrw mrw 84 Sep 14 04:32 check.sh
drwxrwxr-x 2 mrw mrw 4096 Sep 14 04:32 dist
-rwxrwxr-x 1 mrw mrw 446 Sep 14 04:32 pyproject.toml
drwxrwxr-x 4 mrw mrw 4096 Sep 14 04:32 src
drwxrwxr-x 3 mrw mrw 4096 Sep 14 04:32 test
drwxrwxr-x 5 mrw mrw 4096 Sep 14 04:32 venv
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test$ cd q16
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q16$ git init
提示：使用 'master' 作为初始分支的名称。这个默认分支名称可能会更改。要在新仓库中
提示：配置使用初始分支名，并消除这条警告，请执行：
提示：
提示： git config --global init.defaultBranch <名称>
提示：
提示：除了 'master' 之外，通常选定的名字有 'main'、'trunk' 和 'development'。
提示：可以通过以下命令重命名刚创建的分支：
提示：
提示： git branch -m <name>
在 /home/mrw/sys-dev-test/q16/.git/ 初始化空的 Git 仓库
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q16$ git add .
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q16$ git commit -m "Initial commit from q13"
[master (根提交) 53f6aa1] Initial commit from q13
1046 files changed, 192205 insertions(+)
create mode 100755 ai_log.md
create mode 100755 check.sh
create mode 100755 dist/greetlab_25020007122-0.1.0-py3-none-any.whl
create mode 100755 dist/greetlab_25020007122-0.1.0.tar.gz
create mode 100755 dist/greetlab_25020007122-0.1.1-py3-none-any.whl
create mode 100755 dist/greetlab_25020007122-0.1.1.tar.gz
create mode 100755 pyproject.toml
create mode 100644 src/greetlab/__init__.py
create mode 100755 src/greetlab/__pycache__/__init__.cpython-313.pyc
create mode 100755 src/greetlab/__pycache__/test_cli.cpython-313-pytest-9.1.1.pyc
create mode 100644 src/greetlab/cli.py
create mode 100755 src/greetlab_25020007122.egg-info/PKG-INFO
```

图 13: 目录复制与仓库初始化

2.4.2 手动制造错误并复现测试失败

临时修改 `src/greetlab/cli.py`，将问候语改为始终返回字面量“Hello, name!”。执行 `check.sh`，确认测试失败。随后查看错误实现并改正为正确的 f-string 格式。

```
(venv) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q1c$ vim src/greetlab/cli.py
(venv) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q1c$ cat src/greetlab/cli.py
# src/greetlab/cli.py
import argparse

def main():
    p = argparse.ArgumentParser()
    p.add_argument("--name", required=True)
    a = p.parse_args()
    if not a.name.strip():
        raise SystemExit(2)
    print("Hello, name!")
(venv) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q1c$
```

图 14: 手动制造错误

```

Successfully built greetlab_25020007122-0.1.1.tar.gz and greetlab_25020007122-0.1.1-py3-none-any.whl
(venv) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q16$ pip install dist/greetlab_25020007122-0.1.1-py3-none-any.whl
Processing ./dist/greetlab_25020007122-0.1.1-py3-none-any.whl
greetlab-25020007122 is already installed with the same version as the provided wheel. Use --force-reinstall to force an installation of the wheel.
(venv) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q16$ pip install --force-reinstall dist/greetlab_25020007122-0.1.1-py3-none-any.whl
Processing ./dist/greetlab_25020007122-0.1.1-py3-none-any.whl
Installing collected packages: greetlab-25020007122
  Attempting uninstall: greetlab-25020007122
    Found existing installation: greetlab-25020007122 0.1.1
    Uninstalling greetlab-25020007122-0.1.1:
      Successfully uninstalled greetlab-25020007122-0.1.1
Successfully installed greetlab-25020007122-0.1.1
(venv) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q16$ ./check.sh
4 files already formatted
All checks passed!
===== test session starts =====
platform linux -- Python 3.12.3, pytest-9.1.1, pluggy-1.6.0
rootdir: /home/mrw/sys-dev-test/q16
configfile: pyproject.toml
testpaths: test
collected 2 items

test/test_cli.py .F [100%]

===== FAILURES =====
_____ test_normal_name _____

    def test_normal_name():
        cmd = ["sdt-greet", "--name", "王俊杰"]
        result = subprocess.run(cmd, capture_output=True, text=True, check=False)
        assert result.returncode == 0
>       assert "Hello, 王俊杰" in result.stdout
E       AssertionError: assert 'Hello, 王俊杰' in 'Hello, name!\n'
E       + where 'Hello, name!\n' = CompletedProcess(args=['sdt-greet', '--name', '王俊杰'], returncode=0, stdout='Hello, name!\n', stderr='').stdout

test/test_cli.py:15: AssertionError
===== short test summary info =====
FAILED test/test_cli.py::test_normal_name - AssertionError: assert 'Hello, 王俊杰' in 'Hello, name!\n'
===== 1 failed, 1 passed in 0.08s =====
(venv) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q16$

```

图 15: 执行 check.sh 的结果（测试失败）

```

(venv) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q16$ vim src/greetlab/cli.py
(venv) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q16$ cat src/greetlab/cli.py
# src/greetlab/cli.py
import argparse

def main():
    p = argparse.ArgumentParser()
    p.add_argument("--name", required=True)
    a = p.parse_args()
    if not a.name.strip():
        raise SystemExit(2)
    print(f"Hello, {a.name}!")
(venv) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q16$

```

图 16: 改正错误

2.4.3 编写 Makefile

编写包含 check、build 和 clean 的最小 Makefile，其中 check 调用 check.sh，build 生成 wheel。

```

(venv) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q16$ vim Makefile
(venv) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q16$ cat Makefile
.PHONY: check build clean

check:
    python -m build >0
    pip install --force-reinstall dist/greetlab_25020007122-0.1.1-py3-none-any.whl
    ./check.sh
    pip uninstall -y greetlab_25020007122
    rm -rf dist

build:
    python -m build

clean:
    rm -rf dist build *.egg-info
(venv) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q16$

```

图 17: 编写 Makefile

2.4.4 运行 make check 和 make build

执行 make check 和 make build，验证检查通过并成功构建 wheel。

```

(venv) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q16$ make check
python -m build >0
* Creating isolated environment: venv+pip...
* Installing packages in isolated environment:
  - setuptools>=68
* Getting build dependencies for sdist...
* Installed build dependency versions:
  - setuptools==84.0.0
* Building sdist...
warning: sdist: standard file not found: should have one of README, README.rst, README.txt, README.md

* Building wheel from sdist
* Creating isolated environment: venv+pip...
* Installing packages in isolated environment:
  - setuptools>=68
* Getting build dependencies for wheel...
* Installed build dependency versions:
  - setuptools==84.0.0
* Building wheel...
pip install --force-reinstall dist/greetlab_25020007122-0.1.1-py3-none-any.whl
Processing ./dist/greetlab_25020007122-0.1.1-py3-none-any.whl
Installing collected packages: greetlab-25020007122
Successfully installed greetlab-25020007122-0.1.1
./check.sh
4 files already formatted
All checks passed!
===== test session starts =====
platform linux -- Python 3.12.3, pytest-9.1.1, pluggy-1.6.0
rootdir: /home/mrw/sys-dev-test/q16
configfile: pyproject.toml
testpaths: test
collected 2 items

test/test_cli.py .. [100%]

===== 2 passed in 0.05s =====
所有检查通过
pip uninstall -y greetlab_25020007122
Found existing installation: greetlab-25020007122 0.1.1
Uninstalling greetlab-25020007122-0.1.1:
  Successfully uninstalled greetlab-25020007122-0.1.1
rm -rf dist
(venv) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q16$ make build
python -m build
* Creating isolated environment: venv+pip...
* Installing packages in isolated environment:
  - setuptools>=68
* Getting build dependencies for sdist...
running egg_info
writing src/greetlab_25020007122.egg-info/PKG-INFO

```

图 18: 用 make 进行 check 和 build

2.4.5 计算哈希并提交

使用 `sha256sum` 计算 wheel 的 SHA-256 值，编辑 `.gitignore` 忽略生成物，最后将改动提交为内容聚焦的 Git 提交。

```

(venv) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q16$ sha256sum dist/*.whl
7fddd0bde75ebe26a94195cac226b43ad58258b088c53d9d41d2c6efbae2c4a dist/greetlab_25020007122-0.1.1-py3-none-any
.whl
(venv) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q16$ vim .gitignore
(venv) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q16$ cat .gitignore
# Python 字节码缓存
__pycache__/
*.py[cod]
*$py.class

# 构建产物
build/
dist/
*.egg-info/
*.egg

# 测试与覆盖率
.pytest_cache/
.coverage
htmlcov/

# 虚拟环境
venv/
.env/
env/
(venv) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q16$ git branch -m main
(venv) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q16$ git add .
(venv) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q16$ git commit -m "Fix greeting and add Makefile for ch
eck/build"
[main 92fibaa] Fix greeting and add Makefile for check/build
9 files changed, 111 insertions(+), 3 deletions(-)
create mode 100644 .gitignore
create mode 100644 0
create mode 100644 Makefile
delete mode 100755 dist/greetlab_25020007122-0.1.0-py3-none-any.whl
delete mode 100755 dist/greetlab_25020007122-0.1.0.tar.gz
mode change 100755 => 100644 dist/greetlab_25020007122-0.1.1-py3-none-any.whl
mode change 100755 => 100644 dist/greetlab_25020007122-0.1.1.tar.gz
mode change 100755 => 100644 src/greetlab_25020007122.egg-info/PKG-INFO
(venv) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q16$

```

图 19: 计算构建产物的 SHA-256、编辑.gitignore 并提交

3 课后练习

3.1 第 01 题

3.1.1 题目描述

试着写一个正则表达式，并使用 `grep` 命令行工具在你的代码中查找 `subprocess.Popen(..., shell=True)` 的出现位置。然后，试着“破坏”这个 `regex` 模式。`semgrep` 是否仍然能成功匹配到那些会让你的 `grep` 调用失效的危险代码？

3.1.2 解题步骤

在 01 目录下新建 test.py 并写入 `subprocess.Popen('echo test', shell=True)`, 用 `grep -rn "subprocess\.Popen\(.shell=True\)"` . 可以匹配到该调用所在的第 2 行。随后改写 test.py: 把 `subprocess.Popen` 赋给变量 `a`、把 `True` 赋给变量 `b`, 再以 `a('echo test', shell=b)` 的形式调用。程序仍能执行并输出 `test`, 但同样的 `grep` 命令无输出, 说明 `grep` 只做文本模式匹配, 通过使用赋值等方式改写程序结构可以绕过检测。改用 `semgrep` 的 `python.lang.security.audit.subprocess-shell-true` 规则时提示该规则路径不存在, 于是改用 `semgrep -config=auto` . 从 registry 加载规则扫描; `semgrep` 基于抽象语法树分析语义, 因此仍能匹配到这类被改写后的危险调用。

```
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week04$ mkdir 01
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week04$ cd 01
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week04/01$ vim test.py
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week04/01$ cat test.py
import subprocess
subprocess.Popen('echo test', shell=True)
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week04/01$ grep -rn "subprocess\.Popen\(.*shell=True\)" .
./test.py:2:subprocess.Popen('echo test', shell=True)
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week04/01$ vim test.py
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week04/01$ cat test.py
import subprocess
a = subprocess.Popen
b = True
a('echo test', shell=b)
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week04/01$ python3 test.py
test
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week04/01$ grep -rn "subprocess\.Popen\(.*shell=True\)" .
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week04/01$ semgrep --config=python.lang.security.audit.subprocess-shell-true .

Semgrep CLI

Loading rules...
ERROR WARNING: unable to find a config; path 'python.lang.security.audit.subprocess-shell-true' does not exist
ERROR invalid configuration file found (1 configs were invalid)
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week04/01$ semgrep --config=auto .

Semgrep CLI

Loading rules from registry...
Scanning 1 file (only git-tracked) with:

✓ Semgrep OSS
  ✓ Basic security coverage for first-party code vulnerabilities.

✗ Semgrep Code (SAST)
  ✗ Find and fix vulnerabilities in the code you write with advanced scanning and expert security rules.

✗ Semgrep Supply Chain (SCA)
  ✗ Find and fix the reachable vulnerabilities in your OSS dependencies.
```

图 20: grep 匹配危险调用及其被绕过, semgrep 的语义扫描

3.2 第 02 题

3.2.1 题目描述

在你的 IDE 或文本编辑器里练习 regex 查找替换,把这些讲义(https://raw.githubusercontent.com/missing-semester/missing-semester/refs/heads/master/_2026/code-quality.md)中的 - Markdown 项目符号标记替换为 * 项目符号标记。注意,直接替换文件中

所有的 - 字符是不正确的，因为该字符在文件中还有很多并非项目符号标记的用途。

3.2.2 解题步骤

在 02 目录下用 curl 下载讲义 code-quality.md，再在 vim 中使用正则替换：`%s/^-\\s*/ * /`，把行首的 - 项目符号替换为 *，共完成 28 处替换。由于 - 在文中还充当减号与分隔线，直接替换所有 - 会误改正文，故使用 ^ 匹配行首进行避免。

```
nrw@nrw-VMware-Virtual-Platform:~/sys-dev-test/week04$ mkdir 02
nrw@nrw-VMware-Virtual-Platform:~/sys-dev-test/week04$ cd 02
nrw@nrw-VMware-Virtual-Platform:~/sys-dev-test/week04/02$ curl -O https://raw.githubusercontent.com/missing-semester/missing-semester/refs/heads/master/_2026/code-quality.md
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left   Speed
100 19308  100 19308    0     0  18517      0  0:00:01  0:00:01 --:--:-- 18529
nrw@nrw-VMware-Virtual-Platform:~/sys-dev-test/week04/02$ vim code-quality.md
```

图 21: 下载 code-quality.md 并进入 vim 编辑

```

You can find a comprehensive guide to regex syntax in \[this documentation\] (https://docs.python.org/3/library/re.html#regular-expression-syntax) (or one of many other resources available online). Here are some of the basic building blocks:

* `abc` matches the literal string, when the characters have no special meaning (in this example, "abc")
* `.` matches any single character
* `[abc]` matches a single character contained in the brackets (in this example, "a", "b", or "c")
* `[^abc]` matches a single character except those contained in the brackets (e.g., "d")
* `[a-f]` matches a single character contained in the range indicated in the brackets (e.g., "c", but not "q")
* `a|b` matches either pattern (e.g., "a" or "b")
* `\d` matches any digit character (e.g., "3")
* `\w` matches any word character (e.g., "x")
* `\b` matches any word boundary (e.g., in the string "missing semester", matches just before the "m", just after the "g", just before the "s", and just after the "r")
* `(...)` matches the group of a pattern
* `...?` matches zero or one of a pattern, such as `words?` to match "word" or "words"
* `...*` matches any number of a pattern, such as `.*` to match any number of any character
* `...+` matches one or more of a pattern, such as `\d+` to match any non-zero number of digits
* `...{N}` matches exactly N of a pattern, such as `\d{4}` for 4 digits
* `\. ` matches a literal "."
* `\\ ` matches a literal "\"
* `^` matches the start of the line
* `$` matches the end of the line

## Capture groups and references

If you use regex groups `(...)`, you can refer to sub-parts of the match for extraction or search-and-replace purposes. For example, to extract just the month from a YYYY-MM-DD style date, you can use the following Python code:

```python
>>> import re
>>> re.match(r"\d{4}-\d{2}-\d{2}", "2026-01-14").group(1)
'01'
```

In your text editor, you can use reference capture groups in replace patterns. The syntax might vary between IDEs. For example, in VS Code, you can use variables like `$1`, `$2`, etc., and in Vim, you can use `%1`, `%2`, etc., to reference groups.

## Limitations

\[Regular languages\] (https://en.wikipedia.org/wiki/Regular_language) are powerful but limited; there are classes of strings that cannot be expressed as a standard regex (e.g., it is \[not possible\] (https://en.wikipedia.org/wiki/Pumping_lemma_for_regular_languages) to write a regular expression that matches the set of strings {a^n b^n | n ≥ 0}, the set of strings of a number of "a"s followed by the same number of "b"s; more practically, languages like HTML are not regular languages). In practice, modern regex engines support features like look ahead and backreferences that extend support beyond regular languages, and they are practically extremely useful, but it is important to know that they are still limited in their expressive power. For more sophisticated languages, you might need to reach for a more capable type of parser (for one example, see \[pyparsing\] (https://github.com/pyparsing/pyparsing), a \[PEG\] (https://en.wikipedia.org/wiki/Parsing_expression_grammar) parser).
28 行 28 次替换 127,1 82%

```

图 22: 替换后的 Markdown 文件 (共 28 处替换)

3.3 第 03 题

3.3.1 题目描述

写一个 regex, 从形如"name": "Alyssa P. Hacker", "college": "MIT" 的 JSON 结构中捕获 name (例如本例中的 Alyssa P. Hacker)。提示: 在第一次尝试时, 你可能最终会写出一个提取 Alyssa P. Hacker", "college": "MIT 的 regex; 读一读 Python regex 文档中关于贪婪量词的内容, 弄清楚如何修复它。

3.3.2 解题步骤

在 03 目录下新建 test.py, 用 `re.search(r'"name"\s*:\s*"([^\"]*)"', raw).group(1)` 从 JSON 字符串中提取 name。捕获组使用 `[^"]*` 只匹配引号以内的非引号字符, 避免了贪婪量词 `*` 越过第一个引号一直匹配到末尾的问题。执行 `python3 test.py` 输出 Alyssa P. Hacker; 此处由于系统配置问题使用 `python3`。

```
mrw@mrw-Virtual-Platform:~/sys-dev-test/week04/03$ vim test.py
mrw@mrw-Virtual-Platform:~/sys-dev-test/week04/03$ cat test.py
import re
raw = '{"name": "Alyssa P. Hacker", "college": "MIT"}'
result = re.search(r'"name"\s*:\s*"([^\"]*)"', raw).group(1)
print(result)
mrw@mrw-Virtual-Platform:~/sys-dev-test/week04/03$ python test.py
找不到命令 "python", 您的意思是:
  "python3" 命令来自 Debian 软件包 python3
  "python" 命令来自 Debian 软件包 python-is-python3
mrw@mrw-Virtual-Platform:~/sys-dev-test/week04/03$ python3 test.py
Alyssa P. Hacker
mrw@mrw-Virtual-Platform:~/sys-dev-test/week04/03$
```

图 23: 用非贪婪捕获组提取 JSON 中的 name

3.4 第 04 题

3.4.1 题目描述

让第 3 题所写的这个 regex 模式在 name 中包含 " 字符时也能工作 (在 JSON 中, 双引号可以用转义)。

3.4.2 解题步骤

在上题的基础上把捕获组改为“name”\s*:\s*((?:[^\\"\\]|\\.)*): 其中 (?:[^\\"\\]|\\.)* 表示 “一个非引号且非反斜杠的字符” 或 “一个反斜杠后跟任意字符” 的序列，因此遇到被转义的 \ 时不会提前终止匹配。以 raw = r'{"name": "Alyssa P. \Hacker", "college": "MIT"}' 为输入，执行 python3 test.py 输出 Alyssa P. \Hacker。

```
nrw@nrw-Virtual-Platform:~/sys-dev-test/week04/04$ vim test.py
nrw@nrw-Virtual-Platform:~/sys-dev-test/week04/04$ cat test.py
import re
raw = r'{"name": "Alyssa P. \Hacker", "college": "MIT"}'
result = re.search(r'"name":\s*:\s*((?:[^\\"\\]|\\.)*)"', raw).group(1)
print(result)
nrw@nrw-Virtual-Platform:~/sys-dev-test/week04/04$ python3 test.py
Alyssa P. \Hacker
nrw@nrw-Virtual-Platform:~/sys-dev-test/week04/04$
```

图 24: 兼容转义引号的正则表达式

3.5 第 05 题

3.5.1 题目描述

写一个命令程序，从标准输入读取第三题上述形式的 JSON 结构，并把 name 输出到标准输出。你应该只需要几行代码就能完成。在 Python 中，除了 import json 之外，一行代码就可以轻松做到。

3.5.2 解题步骤

在 05 目录下新建 test.py，仅需两行代码：先 import json, sys，再用 print(json.load(sys.stdin)['name']) 从标准输入读取 JSON 并输出 name。将第 3 题形式的 JSON 传入，执行 python3 test.py 输出 Alyssa P. Hacker。可以发现直接调用 json 模块解析相较于手写正则表达式更可靠。

```
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week04/05$ vim test.py
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week04/05$ cat test.py
import json, sys
print(json.load(sys.stdin)['name'])
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week04/05$ python3 test.py
{"name": "Alyssa P. Hacker", "college": "MIT"}
Alyssa P. Hacker
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week04/05$
```

图 25: 从标准输入读取 JSON 并输出 name

3.6 第 06 题

3.6.1 题目描述

大多数的 makefiles 都提供了一个名为 clean 的构建目标，这并不是说我们会生成一个名为 clean 的文件，而是我们可以使用它清理文件，让 make 重新构建。您可以理解为它的作用是“撤销”所有构建步骤。在上面（2020 年课程资料的“元编程”的示例）的 makefile 中为 paper.pdf 实现一个 clean 目标。您需要将构建目标设置为 phony。您也许会发现 git ls-files 子命令很有用。

3.6.2 解题步骤

首先根据课程讲义补齐构建所需的文件。Makefile 中 paper.pdf 依赖 paper.tex 与 plot-data.png，中间产物 plot-data.png 由 plot.py 从 data.dat 生成，故先给 plot.py 添加执行权限并用 git init 记录源文件。随后为 paper.pdf 添加 clean 目标并声明为 phony。执行 clean 命令时先用 rm -f paper.pdf 删除最终产物，再用 git ls-files -others --exclude-standard 列出所有未被 Git 跟踪且未被忽略的文件（即构建产生的中间文件）并一并删除。执行 make paper.pdf 成功生成产物，ls 与 make clean 验证 clean 目标是否只删除产物而保留源文件。

```

mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week04$ mkdir 06
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week04$ cd 06
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week04/06$ vim Makefile
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week04/06$ cat Makefile
paper.pdf: paper.tex plot-data.png
    pdflatex paper.tex

plot-%.png: %.dat plot.py
    ./plot.py -i $*.dat -o $@
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week04/06$ vim paper.tex
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week04/06$ cat paper.tex
\documentclass{article}
\usepackage{graphicx}
\begin{document}
\includegraphics[scale=0.65]{plot-data.png}
\end{document}
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week04/06$ vim plot.py
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week04/06$ cat plot.py
#!/usr/bin/env python
import matplotlib
import matplotlib.pyplot as plt
import numpy as np
import argparse

parser = argparse.ArgumentParser()
parser.add_argument('-i', type=argparse.FileType('r'))
parser.add_argument('-o')
args = parser.parse_args()

data = np.loadtxt(args.i)
plt.plot(data[:, 0], data[:, 1])
plt.savefig(args.o)
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week04/06$ vim data.dat
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week04/06$ cat data.dat
1 1
2 2
3 3
4 4
5 8

```

图 26: 创建 Makefile、paper.tex、plot.py 与 data.dat

```
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week04/06$ vim Makefile
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week04/06$ git init
提示：使用 'master' 作为初始分支的名称。这个默认分支名称可能会更改。要在新仓库中
提示：配置使用初始分支名，并消除这条警告，请执行：
提示：
提示： git config --global init.defaultBranch <名称>
提示：
提示：除了 'master' 之外，通常选定的名字有 'main'、'trunk' 和 'development'。
提示：可以通过以下命令重命名刚创建的分支：
提示：
提示： git branch -m <name>
在 /home/mrw/sys-dev-test/week04/06/.git/ 初始化空的 Git 仓库
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week04/06$ git branch -m main
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week04/06$ git add Makefile paper.tex plot.py data.dat
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week04/06$ git commit -m 'Initail commit: track src'
[main (根提交) dfef1e5] Initail commit: track src
4 files changed, 29 insertions(+)
create mode 100644 Makefile
create mode 100644 data.dat
create mode 100644 paper.tex
create mode 100644 plot.py
```

图 27: 添加 clean 目标并初始化 Git 仓库

```

mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week04/06$ chmod +x plot.py
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week04/06$ vim Makefile
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week04/06$ make paper.pdf
./plot.py -i data.dat -o plot-data.png
pdflatex paper.tex
This is pdfTeX, Version 3.141592653-2.6-1.40.29 (TeX Live 2026) (preloaded format=pdflatex)
 restricted \write18 enabled.
entering extended mode
(./paper.tex
LaTeX2e <2025-11-01>
L3 programming layer <2026-01-19>
(/usr/local/texlive/2026/texmf-dist/tex/latex/base/article.cls
Document Class: article 2025/01/22 v1.4n Standard LaTeX document class
(/usr/local/texlive/2026/texmf-dist/tex/latex/base/size10.clo))
(/usr/local/texlive/2026/texmf-dist/tex/latex/graphics/graphicx.sty
(/usr/local/texlive/2026/texmf-dist/tex/latex/graphics/keyval.sty)
(/usr/local/texlive/2026/texmf-dist/tex/latex/graphics/graphics.sty
(/usr/local/texlive/2026/texmf-dist/tex/latex/graphics/trig.sty)
(/usr/local/texlive/2026/texmf-dist/tex/latex/graphics-cfg/graphics.cfg)
(/usr/local/texlive/2026/texmf-dist/tex/latex/graphics-def/pdftex.def)))
(/usr/local/texlive/2026/texmf-dist/tex/latex/l3backend/l3backend-pdftex.def)
No file paper.aux.
(/usr/local/texlive/2026/texmf-dist/tex/context/base/mkii/supp-pdf.mkii
[Loading MPS to PDF converter (version 2006.09.02).]
) (/usr/local/texlive/2026/texmf-dist/tex/latex/epstopdf-pkg/epstopdf-base.sty
(/usr/local/texlive/2026/texmf-dist/tex/latex/latexconfig/epstopdf-sys.cfg))
[1{/usr/local/texlive/2026/texmf-var/fonts/map/pdftex/updmap/pdftex.map} <./plot-
data.png>] (./paper.aux) )</usr/local/texlive/2026/texmf-dist/fonts/type1/pub
lic/amsfonts/cm/cmr10.pfb>
Output written on paper.pdf (1 page, 20987 bytes).
Transcript written on paper.log.
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week04/06$ ls -l paper.
ls: 无法访问 'paper.': 没有那个文件或目录
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week04/06$ ls -l paper.pdf
-rw-rw-r-- 1 mrw mrw 20987 Sep 21 02:53 paper.pdf
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week04/06$ make clean
rm -f paper.pdf
rm -f $(git ls-files --others --exclude-standard)
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week04/06$ ls
data.dat Makefile paper.tex plot.py
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week04/06$

```

图 28: make paper.pdf 与 make clean 的执行结果

3.7 第 07 题

3.7.1 题目描述

Git 可以作为一个简单的 CI 系统来使用,在任何 git 仓库中的.git/hooks 目录中,您可以找到一些文件(当前处于未激活状态),它们的作用和脚本一样,当某些事件发生时便可以自动执行。请编写一个 pre-commit 钩子,它会在提交前执行 make paper.pdf 并在出现构建失败的情况拒绝您的提交。

3.7.2 解题步骤

在.git/hooks下新建pre-commit钩子并赋予执行权限，pre-commit先执行make paper.pdf，若构建失败则打印Build failed. Commit rejected.并以退出码1中止提交。用git add .将原先的4个文件加入缓存区随后提交，pre-commit执行构建成功后提交正常完成。再把paper.tex中的图片改为不存在的plot-nonexist.png后再次提交，pdflatex因找不到该文件报错，make返回非零状态码，pre-commit输出Build failed. Commit rejected.并拒绝该次提交。

```

mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week04/06$ vim .git/hooks/pre-commit
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week04/06$ cat .git/hooks/pre-commit
#!/bin/sh
if ! make paper.pdf; then
    echo "Build failed. Commit rejected."
    exit 1
fi
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week04/06$ chmod +x .git/hooks/pre-commit
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week04/06$ ls
data.dat  Makefile  paper.tex  plot.py
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week04/06$ git add .
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week04/06$ git commit -m 'Hook test 1 : All goes well'
./plot.py -i data.dat -o plot-data.png
pdflatex paper.tex
This is pdfTeX, Version 3.141592653-2.6-1.40.29 (TeX Live 2026) (preloaded format=pdflatex)
 restricted \write18 enabled.
entering extended mode
(./paper.tex
LaTeX2e <2025-11-01>
L3 programming layer <2026-01-19>
(/usr/local/texlive/2026/texmf-dist/tex/latex/base/article.cls
Document Class: article 2025/01/22 v1.4n Standard LaTeX document class
(/usr/local/texlive/2026/texmf-dist/tex/latex/base/size10.clo))
(/usr/local/texlive/2026/texmf-dist/tex/latex/graphics/graphicx.sty
(/usr/local/texlive/2026/texmf-dist/tex/latex/graphics/keyval.sty)
(/usr/local/texlive/2026/texmf-dist/tex/latex/graphics/graphics.sty
(/usr/local/texlive/2026/texmf-dist/tex/latex/graphics/trig.sty)
(/usr/local/texlive/2026/texmf-dist/tex/latex/graphics-cfg/graphics.cfg)
(/usr/local/texlive/2026/texmf-dist/tex/latex/graphics-def/pdftex.def)))
(/usr/local/texlive/2026/texmf-dist/tex/latex/l3backend/l3backend-pdftex.def)
No file paper.aux.
(/usr/local/texlive/2026/texmf-dist/tex/context/base/mkii/supp-pdf.mkii
[Loading MPS to PDF converter (version 2006.09.02).]
) (/usr/local/texlive/2026/texmf-dist/tex/latex/epstopdf-pkg/epstopdf-base.sty
(/usr/local/texlive/2026/texmf-dist/tex/latex/latexconfig/epstopdf-sys.cfg))
[1{/usr/local/texlive/2026/texmf-var/fonts/map/pdftex/updmap/pdftex.map}] <./plo
t-data.png>] (./paper.aux) </usr/local/texlive/2026/texmf-dist/fonts/type1/pub
lic/amsfonts/cm/cmr10.pfb>
Output written on paper.pdf (1 page, 20987 bytes).
Transcript written on paper.log.
[main 1944fie] Hook test 1 : All goes well
 2 files changed, 7 insertions(+), 1 deletion(-)
 mode change 100644 => 100755 plot.py
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week04/06$

```

图 29: 编写 pre-commit 钩子并成功提交

```

mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week04/06$ make clean
rm -f paper.pdf
rm -f $(git ls-files --others --exclude-standard)
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week04/06$ vim paper.tex
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week04/06$ cat paper.tex
\documentclass{article}
\usepackage{graphicx}
\begin{document}
\includegraphics[scale=0.65]{plot-nonexist.png}
\end{document}

```

图 30: 清理产物并把图片改为不存在的文件

```

mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week04/06$ git add .
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week04/06$ git commit -m 'Hook test 2 : PNG is nonexistent'
./plot.py -i data.dat -o plot-data.png
pdflatex paper.tex
This is pdfTeX, Version 3.141592653-2.6-1.40.29 (TeX Live 2026) (preloaded format=pdflatex)
 restricted \write18 enabled.
entering extended mode
(./paper.tex
LaTeX2e <2025-11-01>
L3 programming layer <2026-01-19>
(/usr/local/texlive/2026/texmf-dist/tex/latex/base/article.cls
Document Class: article 2025/01/22 v1.4n Standard LaTeX document class
(/usr/local/texlive/2026/texmf-dist/tex/latex/base/size10.clo))
(/usr/local/texlive/2026/texmf-dist/tex/latex/graphics/graphicx.sty
(/usr/local/texlive/2026/texmf-dist/tex/latex/graphics/keyval.sty)
(/usr/local/texlive/2026/texmf-dist/tex/latex/graphics/graphics.sty
(/usr/local/texlive/2026/texmf-dist/tex/latex/graphics/trig.sty)
(/usr/local/texlive/2026/texmf-dist/tex/latex/graphics-cfg/graphics.cfg)
(/usr/local/texlive/2026/texmf-dist/tex/latex/graphics-def/pdftex.def)))
(/usr/local/texlive/2026/texmf-dist/tex/latex/l3backend/l3backend-pdftex.def)
No file paper.aux.
(/usr/local/texlive/2026/texmf-dist/tex/context/base/mkii/supp-pdf.mkii
[Loading MPS to PDF converter (version 2006.09.02).]
) (/usr/local/texlive/2026/texmf-dist/tex/latex/epstopdf/pkg/epstopdf-base.sty
(/usr/local/texlive/2026/texmf-dist/tex/latex/latexconfig/epstopdf-sys.cfg))

LaTeX Warning: File 'plot-nonexist.png' not found on input line 4.

! Package pdftex.def Error: File 'plot-nonexist.png' not found: using draft set
ting.

See the pdftex.def package documentation for explanation.
Type H <return> for immediate help.
...

l.4 ...luidegraphics[scale=0.65]{plot-nonexist.png}

?
! Emergency stop.
...

l.4 ...luidegraphics[scale=0.65]{plot-nonexist.png}

! ==> Fatal error occurred, no output PDF file produced!
Transcript written on paper.log.
make: *** [Makefile:4: paper.pdf] 错误 1
Build failed. Commit rejected.
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week04/06$

```

图 31: 构建失败导致提交被拒绝

3.8 第 08 题

3.8.1 题目描述

学习一下 Rust 的构建系统的依赖管理，对于每种语法 (尖号、波浪号、通配符、比较、多个版本要求)，构建一种场景使其具有实际意义。

3.8.2 解题步骤

- **尖号 (^):** ^1.2.3 等价于 $\geq 1.2.3, < 2.0.0$ 。适用于希望自动获得向后兼容的新功能,例如 ^1.2.3 允许升级到 1.9.0,但不会升级到 2.0.0。
- **波浪号 (~):** ~1.2.3 允许升级到 1.2.x ($x \geq 3$),但不会改变次版本号。适用于需要特定次版本系列内的补丁更新。
- **通配符 (*):** 1.* 匹配所有 1.x.y 版本。适用于内部工具,不关心具体版本,只要主版本号一致即可。
- **比较 (>、<):** >1.2.3 或 <2.0.0。适用于需要严格限定版本范围的场景,例如避开某个已知有问题的版本。
- **多个版本要求:** 用逗号分隔,如 $\geq 1.2.0, < 1.5.0$ 。适用于需要同时满足多个约束条件的场景。

3.9 第 09 题

3.9.1 题目描述

基于 GitHub Pages 创建任意一个可以自动发布的页面。添加一个 GitHub Action 到该仓库,对仓库中的所有 shell 文件执行 shellcheck。

3.9.2 解题步骤

先在仓库设置中将 Pages 的 Source 设为 GitHub Actions,并在仓库根目录放置包含“系统开发工具基础”与“这是一个测试页面”的 index.html,发布后可通过 <https://mrw1114.github.io/fundamentals-of-system-dev-tools/> 访问。随后在 .github/workflows/blank.yml 中新增工作流,在 push 与 pull_request 时检查仓库,并用 ludeeus/action-shellcheck 对仓库中所有 shell 文件执行 shellcheck。首次运行失败,报出 SC2148 (脚本缺少 shebang) 与 SC2086 (变量未加双引号) 等问题,为相关脚本补上 shebang 并修正引号后再次运行,检查全部通过。



图 32: 启用 GitHub Pages 后的测试页面

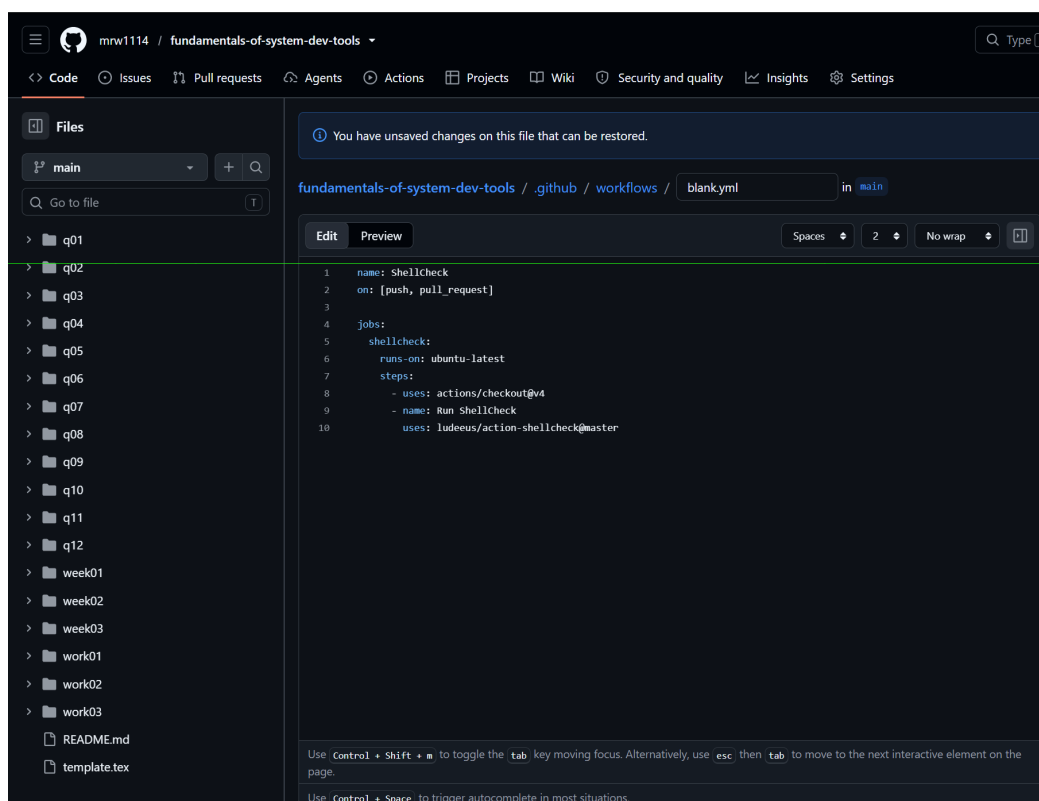


图 33: shellcheck workflow 配置

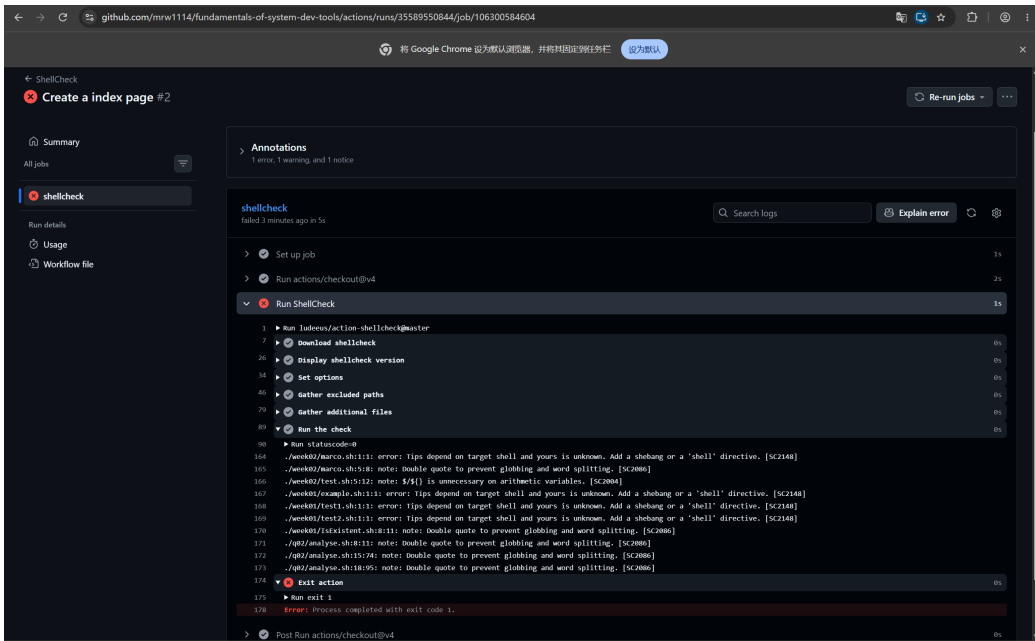


图 34: 首次运行报出 SC2148、SC2086 等问题

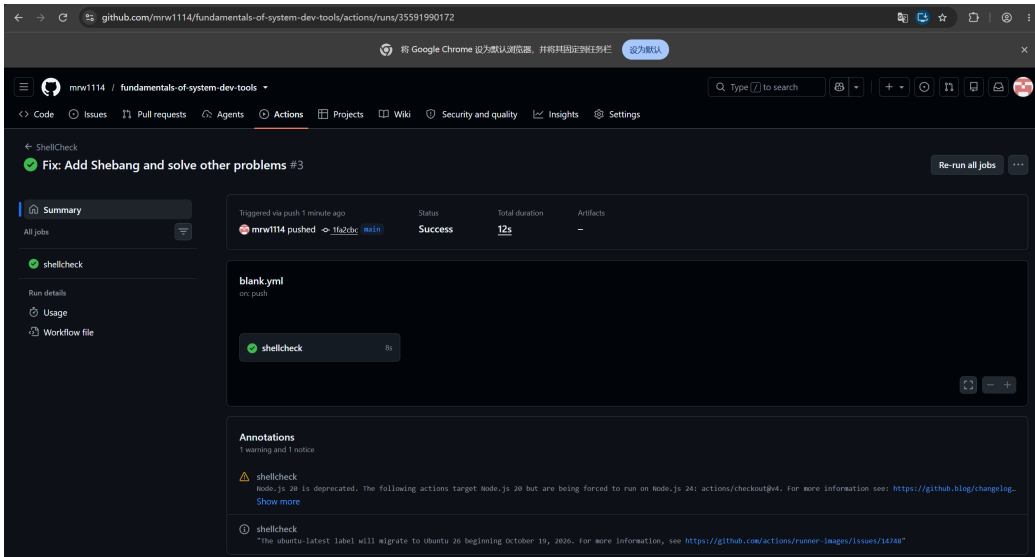


图 35: 补充 shebang 并修正后检查通过

3.10 第 10 题

3.10.1 题目描述

构建属于您的 GitHub action，对仓库中所有的.md 文件执行 proselint 或 write-good，在您的仓库中开启这一功能，提交一个包含错误的文件看看该功能是否生效。

3.10.2 解题步骤

在.github/workflows/proselint.yml 中新增工作流:在 push 与 pull_request 时检出仓库、安装 proselint，并用 proselint check \$(git ls-files "*.md") 对仓库中所有.md 文件执行检查。由于该工具会执行仅涉及写作风格的双引号和省略号检查，因此在仓库根目录添加 proselintrc.json 并设置相应的忽略规则。在提交后该功能生效并报告 README.md 与 week03/ai_summary.md 分别报出 lexical_illusions（词语重复）、annotations.misc（未移除的 TODO 等项目标记）等错误。修复相关问题后，仓库首页显示包含 Proselint 和 ShellCheck 的 5 项检查全部通过。

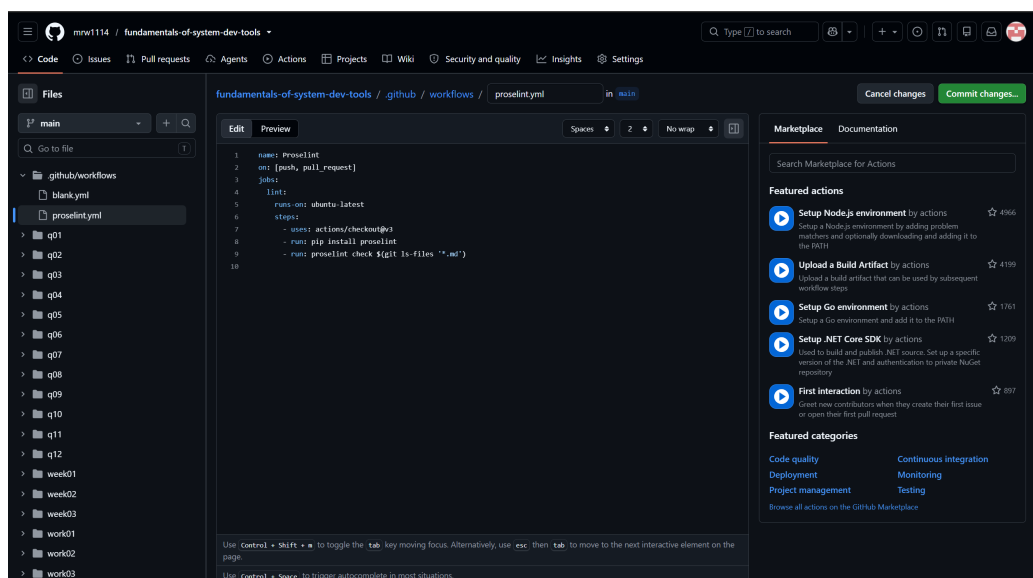


图 36: proselint 工作流程配置

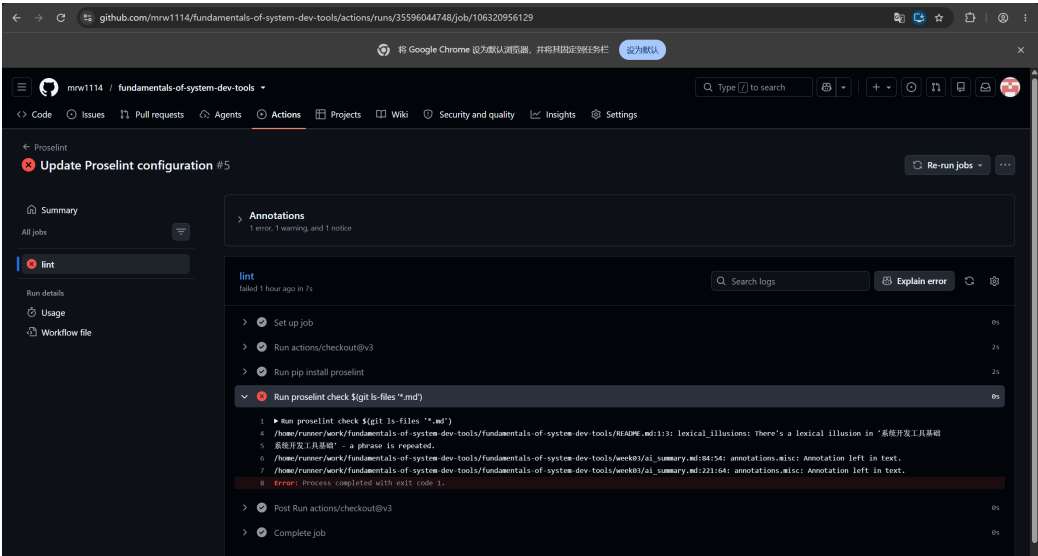


图 37: 提交包含错误的文件后检查失败

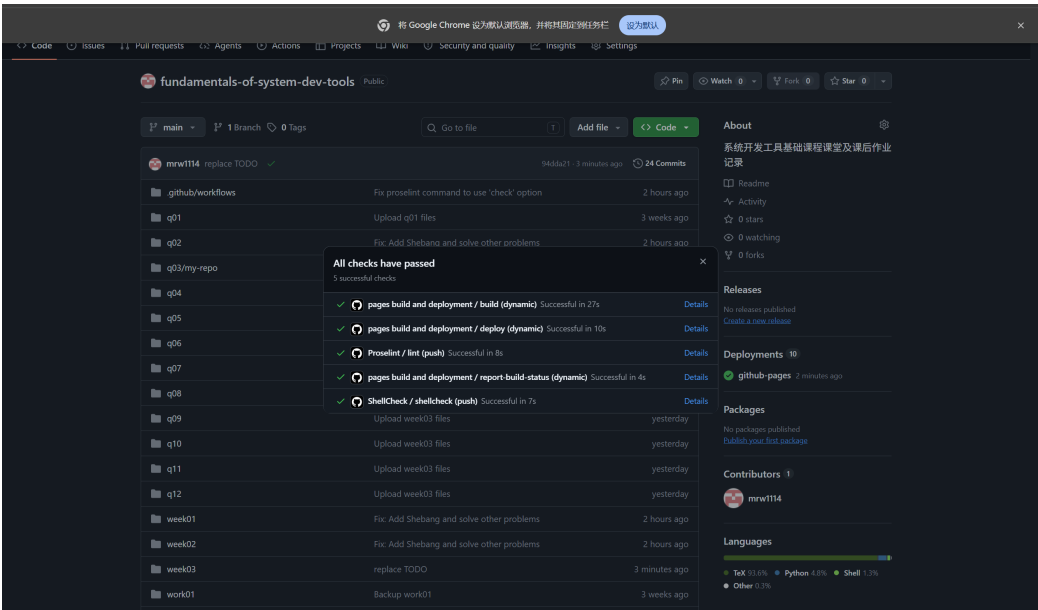


图 38: 修复后全部检查通过

4 commit 记录

commit 记录截图如下，由于课程文件中的 week04/02/code-quality.md 是作业文件但又被 ProseLint 检测，故导致 action 中的检测不通过。后续在配置文件中强行排除该文件后检查通过。详细内容请访问仓库查看。

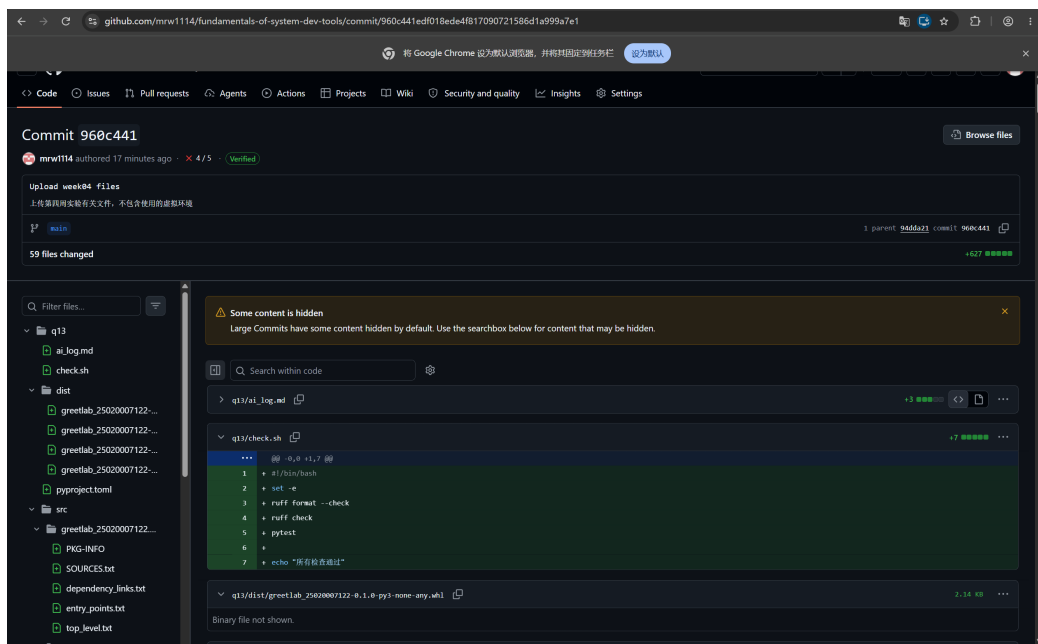


图 39: commit 记录截图